Azure Key Vault — Task 3

Replace a hardcoded secret in a Python script and enable audit logging · Completed 10 November 2026

The HVMC Lab Results Sync job needs an API key. Instead of keeping that key in the script, the script now asks Azure Key Vault for it at run time, and every read of the secret is recorded in Log Analytics.

Prepared By	Sabitha S Karanavar
Date	November 10, 2026
Script	`learning/scripts/lab_results_sync.py` (Python 3.9, azure-identity 1.26, azure-keyvault-secrets 4.11)
Vault / logs	hvmc-accessguard-kv → diagnostic setting `kv-audit-to-law` (AuditEvent) → HVMC-SecOps-LAW
Branch	`feature/azure-key-vault-setup` (learning/README.md Entry 4)
Cost	Under $0.01 (a few KB of logs; workspace capped at 0.03 GB/day)

Summary

#	Step	Status
1	Python environment with azure-identity and azure-keyvault-secrets (local, kept out of git)	✅ Done
2	Script reads **accessguard-api-key** with **DefaultAzureCredential**; no secret in the code	✅ Done
3	Vault diagnostic logging: AuditEvent → Log Analytics workspace HVMC-SecOps-LAW	✅ Done
4	Script run: secret found, value never printed (Figure 1)	✅ Done
5	KQL query shows the **SecretGet** event for the script (Figure 2)	✅ Done
6	Proof that no secret value is in any file or commit of the repository	✅ Done

> **Success criterion (learning/PLAN.md):** the script runs with no secret in the code or repository, and a Log Analytics query shows a `SecretGet` event for the script's identity — **met**.

1. Before and after

```
# lab_results_sync.py  (BEFORE - shown here only as the bad example, never committed)
ACCESSGUARD_API_KEY = "kX9f...real-key-in-the-file..."   # anyone with the file or the repo can read it

def call_lab_results_api():
    headers = {"x-api-key": ACCESSGUARD_API_KEY}
    ...
```

```
# lab_results_sync.py  (AFTER - learning/scripts/lab_results_sync.py)
from azure.identity import DefaultAzureCredential
from azure.keyvault.secrets import SecretClient

VAULT_URL = os.environ.get("KEY_VAULT_URL", "https://hvmc-accessguard-kv.vault.azure.net")
credential = DefaultAzureCredential(exclude_interactive_browser_credential=True)
client = SecretClient(vault_url=VAULT_URL, credential=credential)
secret = client.get_secret("accessguard-api-key")     # read at run time, held in memory only
# the value is never printed or written to disk
```

	Before (hardcoded)	After (Key Vault)
Where the key lives	In the script and in Git	Only in the vault
Who can read it	Anyone with the file	Only identities with a Key Vault data role (Azure RBAC)
Audit trail	None	Every read recorded as SecretGet in Log Analytics
Rotation	Edit and redeploy the code	Add a new secret version; the code does not change

2. How DefaultAzureCredential finds an identity

`DefaultAzureCredential` tries a fixed list of credential types and uses the first one that works, so the same code runs on a laptop and in production without changes:

Where the code runs	Identity it uses	Used here
Developer Mac	Azure CLI login (`az login`)	✅ this task (IAM admin, Key Vault Secrets Officer)
Azure VM, Function, App Service	Managed identity (no credential to store at all)	Recommended for production
Server or pipeline	App registration with a certificate (environment variables)	Possible with HVMC Lab Results Sync (Task 2)

3. Script run

```
$ ~/Downloads/HVMC_KeyVault_Lab/task3/run.sh
[2026-11-10 19:06:19 UTC] AccessGuard Health - HVMC Lab Results Sync
  vault  : https://hvmc-accessguard-kv.vault.azure.net
  secret : accessguard-api-key
  identity source : AzureCliCredential
  secret found    : yes, 32 characters, version ...8c8dca
  secret value    : ******** (never printed or saved)
  lab-results API : would call https://lab-results.hvmc.example/api/v1/results with the key in the request
header (lab simulation)
  RESULT : OK - no secret stored in this script or its folder
```

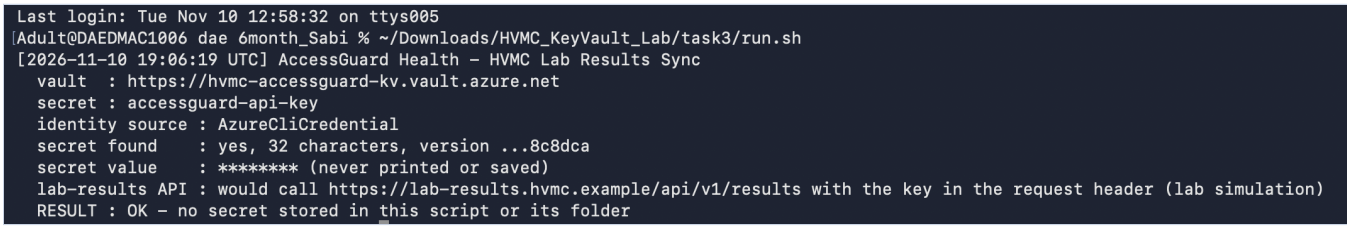


Figure 1. The Lab Results Sync script reads accessguard-api-key from Key Vault with DefaultAzureCredential (Azure CLI identity); the value is never printed.

4. Audit logging to Log Analytics

```
$ az monitor diagnostic-settings create --name kv-audit-to-law --resource <vault-id> \
      --workspace <HVMC-SecOps-LAW-id> --logs '[{"category":"AuditEvent","enabled":true}]'
Name             Logs        Workspace
---------------  ----------  ------------------------------------------------------------
kv-audit-to-law  AuditEvent  .../resourceGroups/HVMC-CloudSecurity-RG/.../workspaces/HVMC-SecOps-LAW
```

Query used in the workspace (it leaves out the caller's e-mail and IP address, which are also in the raw log):

```
AzureDiagnostics
| where ResourceProvider == "MICROSOFT.KEYVAULT" and OperationName == "SecretGet"
| extend SecretName = tostring(split(column_ifexists("id_s", ""), "/")[4])
| extend IdentityType = column_ifexists("identity_claim_idtyp_s", "")
| extend Caller = iff(IdentityType == "app", "Application", "User (IAM admin)")
| extend ci = column_ifexists("clientInfo_s", "")
| extend Client = case(ci has "keyvault-secrets", "lab_results_sync.py (Azure SDK)",
                       ci has "AZURECLI", "Azure CLI command (no-secret check)", ci)
| project TimeGenerated, OperationName, SecretName, Caller, IdentityType, Client, ResultSignature,
httpStatusCode_d
| order by TimeGenerated desc
```

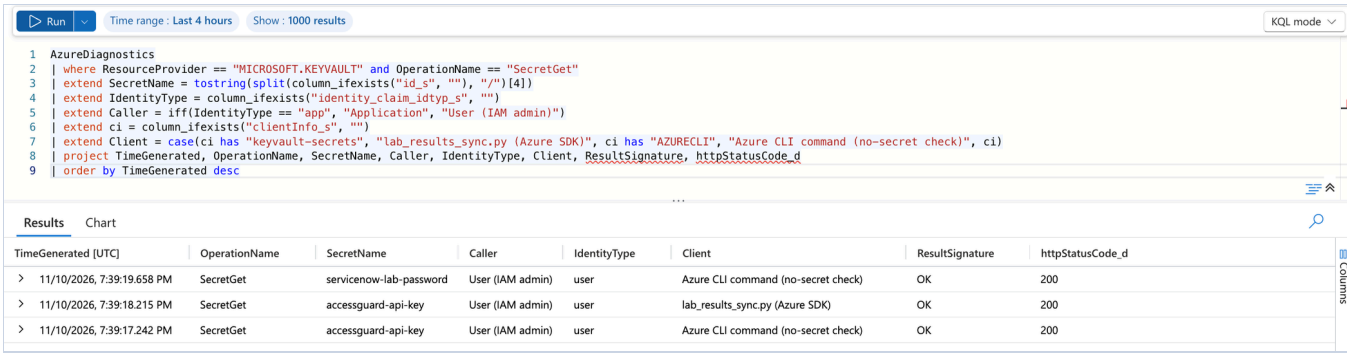


Figure 2. Key Vault audit log in Log Analytics: the SecretGet by lab_results_sync.py (Azure SDK, user identity, OK/200), plus the two reads made by the Azure CLI no-secret check — every secret read is recorded.

Time (UTC)	Secret	Client (from the log)	Meaning
19:39:18	accessguard-api-key	azsdk-python-keyvault-secrets	**The script** reading its API key
19:39:17	accessguard-api-key	AZURECLI	The no-secret check reading the value to search the repo for it
19:39:19	servicenow-lab-password	AZURECLI	The same check for the second secret

5. Proof: no secret in the repository

`check_no_secret.sh` reads each real secret value from the vault into memory, searches every file (`git grep`) and every commit on all branches (`git log --all -S`) for it, and prints only the counts — never the value.

```
$ ./check_no_secret.sh
Repository: dae 6month_Sabi   branch: main
  accessguard-api-key     : files containing the value = 0   commits that ever added it = 0
  servicenow-lab-password : files containing the value = 0   commits that ever added it = 0
Hardcoded-key patterns in learning/scripts:
  none found
```

6. Challenges and how I solved them

Challenge	Cause	Fix
No SecretGet event for over 30 minutes	A brand-new diagnostic setting takes several minutes to start, and the first logs can take much longer to arrive	Checked ingestion and the daily cap were fine, waited, and made a fresh read; the first events arrived after about 35 minutes. The reads made in the first 20 minutes after enabling were never logged
Query error: identity_claim_upn_s not found	The table only gets the Key Vault columns after the first Key Vault event	Used `column_ifexists()` so the query works before and after

Challenge	Cause	Fix
First query labelled the caller “Application”	The user's name is in a different field than expected	Inspected the raw event; used `identity_claim_idtyp_s` (user / app) and `clientInfo_s` to tell the script from Azure CLI commands
A pasted “kusto” line broke the query	The code-block label was copied with the query	Removed it; the query starts with the table name

> **Key lesson.** Audit logs only help if you read them carefully: the first query gave a believable but wrong caller. Checking the raw fields turned it into accurate evidence — and showed that even an administrator's own checks are recorded.

7. Next steps

Item	Why
Run the script as the application (HVMC Lab Results Sync, certificate) or with a managed identity	Applications, not people, should read application secrets
Alert rule on unusual SecretGet activity (e.g. a new caller or many reads)	Turn the audit trail into detection
Secret expiry dates and rotation	Limit how long a leaked key stays useful
Network hardening (private endpoint / firewall) and HashiCorp Vault as the next phase	Beyond this month's scope